

Multiprocessors and Locking

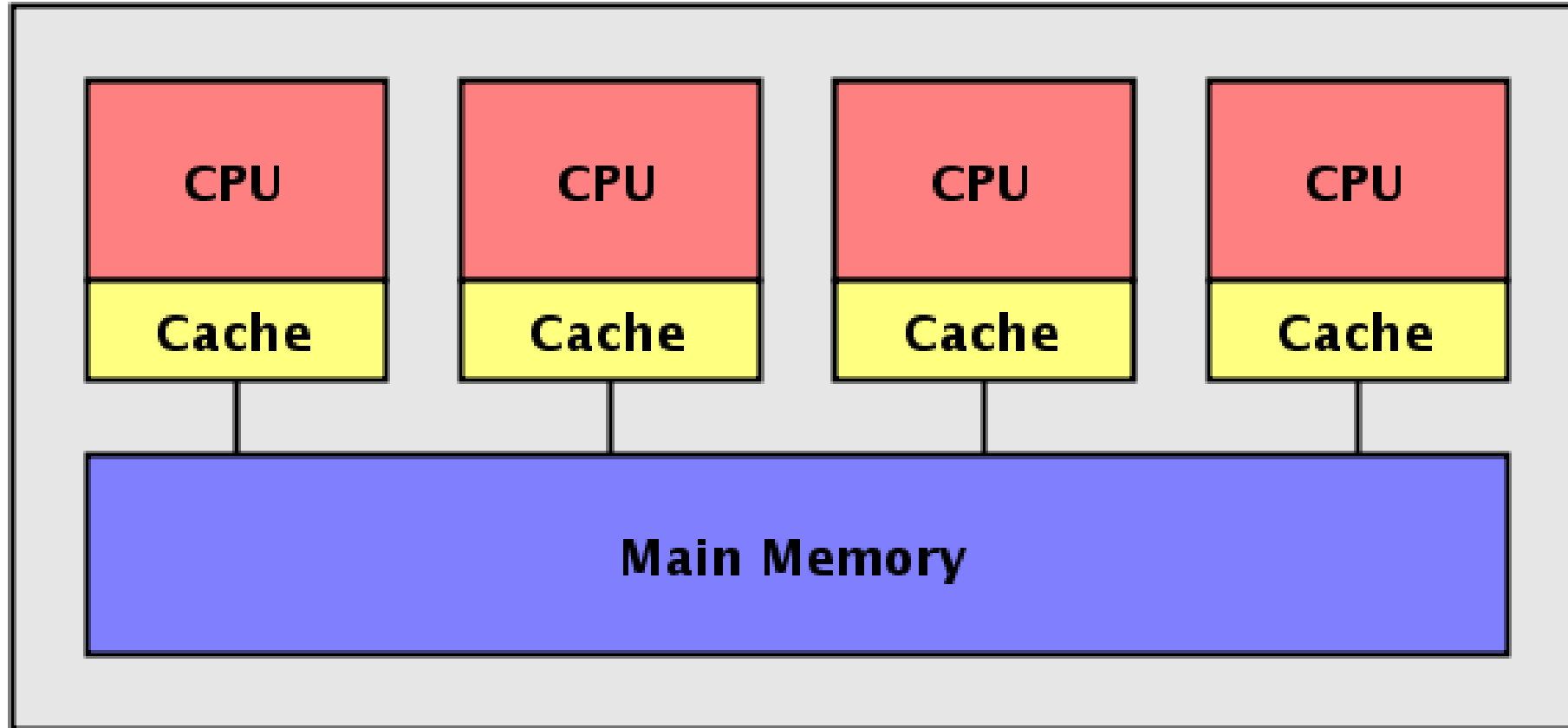
Multiprocessors and Locking

In multiprocessor systems, where multiple processors share resources and execute processes concurrently, locking mechanisms are critical for ensuring data consistency, preventing race conditions, and coordinating access to shared resources.

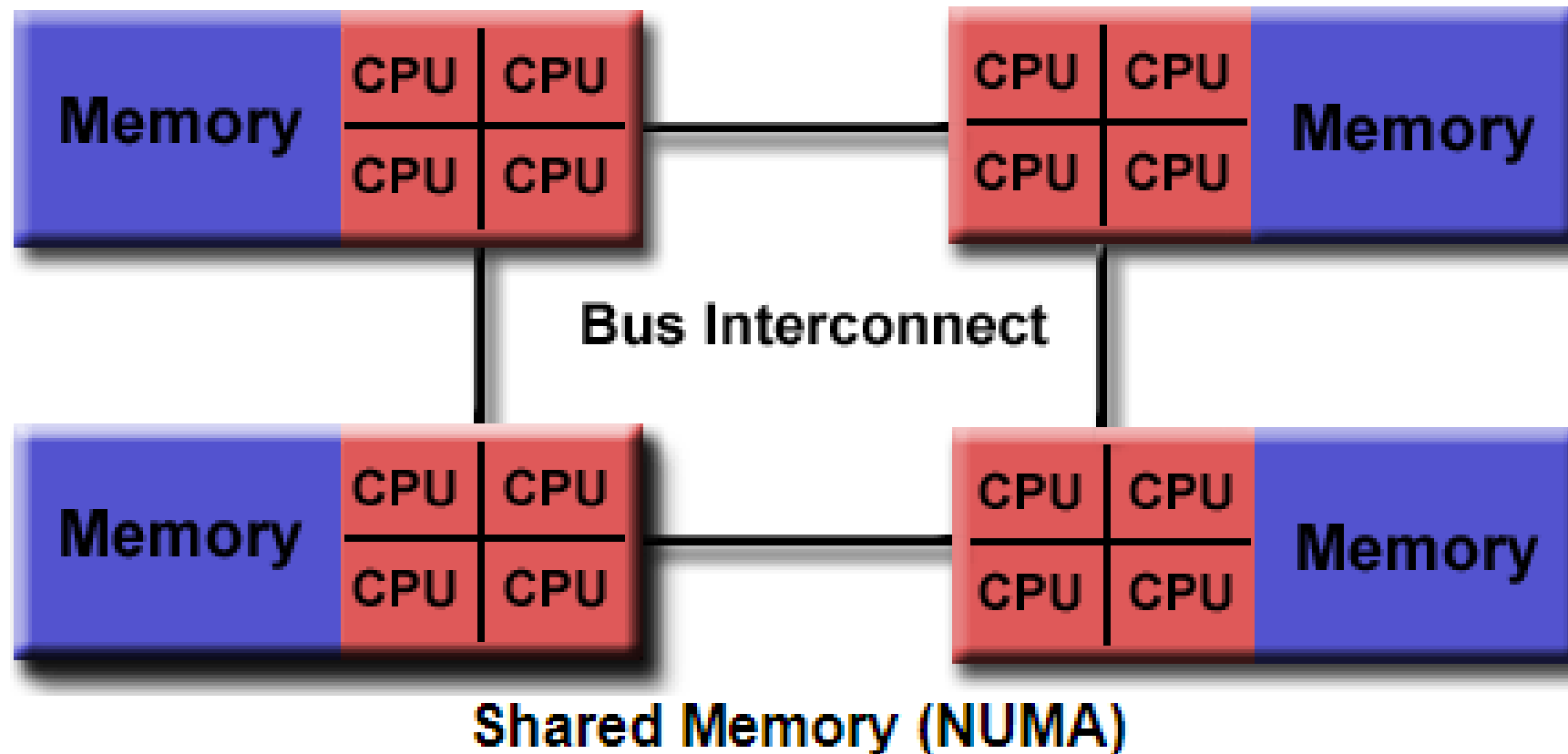
Without proper synchronization, concurrent access can lead to inconsistent states.

Locks are synchronization primitives used to control access to shared resources.

Multiprocessors - Shared Memory Architecture - UMA



Multiprocessors - Shared Memory Architecture - NUMA



Uniform Memory Access (UMA)

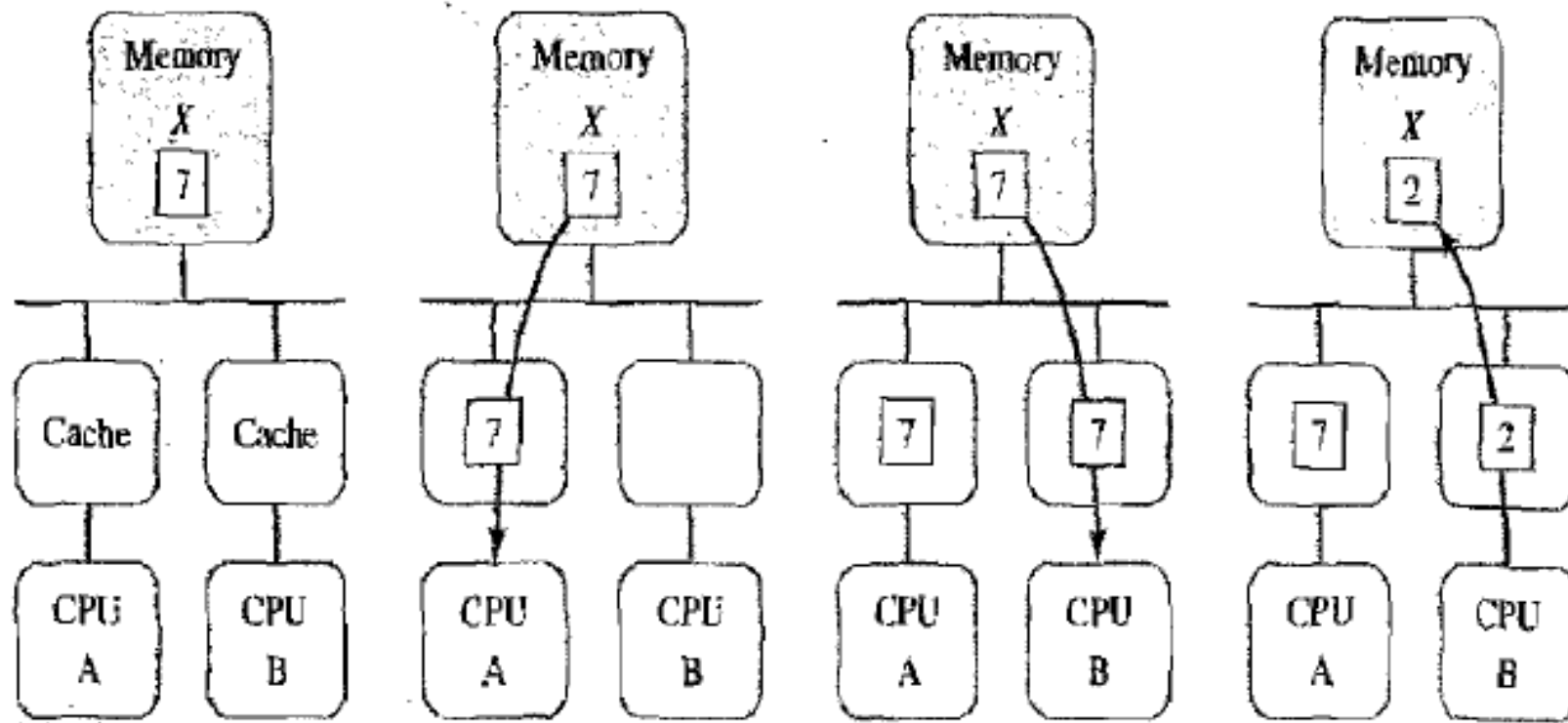


Fig. Cache coherence problem

Uniform Memory Access (UMA)

Cache coherence – data inconsistency problem at caches in multiprocessor system

- Cache Write policies: Write through and write back
- Cache Coherence Schemes:
 - Write through + write back
 - Write invalidate + write update + Write back - policies
- Snooping" protocols are typically used to maintain cache coherence on centralized Multiprocessors.
- Processor Synchronization – Mutual exclusion / barrier synchronization

Locks

- A simple example of a serializing mechanism is the use of exclusive locks.
- Server can lock any object that is about to be used by a client.
- If another client wants to access the same object, it has to wait until the object is unlocked in the end.

Figure: Transactions *T* and *U* with exclusive locks

Transaction <i>T</i> :		Transaction <i>U</i> :	
<i>balance</i> = <i>b.getBalance()</i> <i>b.setBalance(bal*1.1)</i> <i>a.withdraw(bal/10)</i>		<i>balance</i> = <i>b.getBalance()</i> <i>b.setBalance(bal*1.1)</i> <i>c.withdraw(bal/10)</i>	
Operations	Locks	Operations	Locks
<i>openTransaction</i>		<i>openTransaction</i>	
<i>bal</i> = <i>b.getBalance()</i>	lock <i>B</i>	<i>bal</i> = <i>b.getBalance()</i>	waits for <i>T</i> 's lock on <i>B</i>
<i>b.setBalance(bal*1.1)</i>		...	
<i>a.withdraw(bal/10)</i>	Lock <i>A</i>		lock <i>B</i>
<i>closeTransaction</i>	unlock <i>A</i> , <i>B</i>	<i>b.setBalance(bal*1.1)</i>	
		<i>c.withdraw(bal/10)</i>	lock <i>C</i>
		<i>closeTransaction</i>	unlock <i>B</i> , <i>C</i>

Figure: Lock compatibility

For one object		Lock requested	
		read	write
Lock already set	none	OK	OK
	read	OK	wait
	write	wait	wait

An object can be read and write. From the compatibility table, we know pairs of read operations from different transactions do not conflict. So a **simple exclusive lock** used for both read and write **reduces concurrency** more than necessary. (Many readers/Single writer)

Rules;

1. If T has already performed a read operation, then a concurrent transaction U must not write until T commits or aborts.
2. If T already performed a write operation, then concurrent U must not read or write until T commits or aborts.

Shared Lock and Exclusive lock

- Locks must be obtained before read/write can begin
- If a transaction want to read and write the same object, it can either
 - Obtain an **X-lock** before reading and unlock it immediately afterwards
 - Obtain an **S-lock** before reading, then obtain an X-lock before writing. And unlock it immediately afterwards.

Scalable Locks

In operating systems, scalable locks are designed to efficiently handle synchronization in environments with many threads or processors, ensuring performance does not degrade as the system scales.

Traditional locking mechanisms like basic spinlocks or mutexes can become bottlenecks when contention increases, especially in multiprocessor systems.

Key Characteristics of Scalable Locks

1. **Low Contention:** Minimize the number of threads contending for the same lock.
2. **Reduced Overhead:** Avoid excessive CPU usage due to spinning or frequent context switches.
3. **Fairness:** Ensure threads are served in a fair order, avoiding starvation.
4. **Scalability:** Perform well as the number of processors or threads increases.

Scalable Locks - Types

1. Ticket Locks

How It Works:

Each thread receives a unique ticket when requesting a lock.
The lock is granted to the thread with the next ticket number.

Advantages:

Ensures fairness by serving threads in order.
Avoids contention hot-spots since threads spin only when their turn approaches.

2. MCS Locks (Mellor-Crummey and Scott)

How It Works:

Maintains a queue of waiting threads, with each thread spinning on its own memory location rather than a shared one.
Threads pass the lock to the next in the queue.

Advantages:

Reduces contention on shared memory, making it scalable in NUMA (Non-Uniform Memory Access) architectures.

Scalable Locks - Types

3. Read-Write Locks

How It Works:

Allows multiple threads to hold the lock in read mode, but only one thread in write mode.

Advantages:

Improves concurrency for workloads with frequent reads and infrequent writes.

4. Adaptive Locks

How It Works:

Switches between spinning (busy-waiting) and blocking (context switch) based on the lock's contention level.

Advantages:

Balances the trade-off between spinning and blocking to reduce overhead.

Lock-Free Coordination

Lock-free coordination is a technique in concurrent programming where multiple threads or processes interact and manipulate shared data **without using locks**.

Instead of relying on traditional locking mechanisms like mutexes or semaphores, it **uses atomic operations and non-blocking algorithms** to ensure safe, consistent access to shared resources.

Lock-free algorithms are essential in applications requiring extreme parallelism, such as simulations or machine learning.

In real-time systems, locking mechanisms introduce unpredictable delays. Lock-free coordination ensures consistent, low-latency behavior.

Characteristics of Lock-Free Coordination

Non-blocking: Threads are never forced to wait for a lock to be released.

Progress Guarantee: At least one thread makes progress at any time (preventing deadlock).

Minimized Overhead: No context switching due to blocking, reducing latency and improving performance.

Scalability: Performs well under high contention, especially in multiprocessor systems.

Lock-Free Coordination - Techniques

1. Atomic Operations

Definition: Operations performed as a single, indivisible step.

Examples:

Compare-And-Swap (CAS): Compares a value at a memory location with an expected value and updates it if they match.

2. Lock-Free Data Structures

Specially designed data structures that use atomic operations for thread-safe operations.

Examples:

Lock-Free Queues: Threads enqueue and dequeue elements atomically.

Lock-Free Stacks: Allow concurrent push and pop operations.

Lock-Free Hash Tables: Support concurrent inserts, deletes, and lookups.